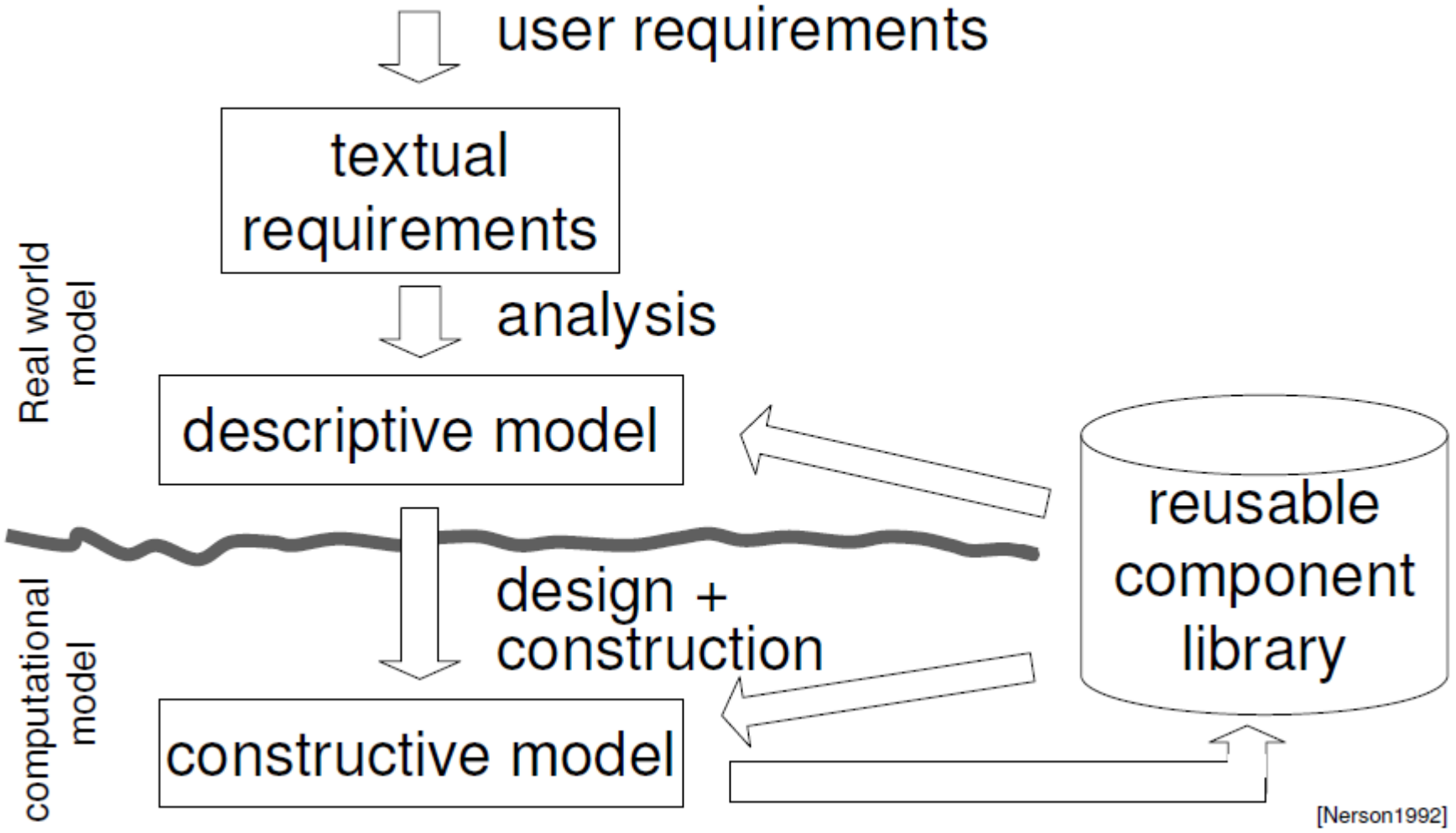
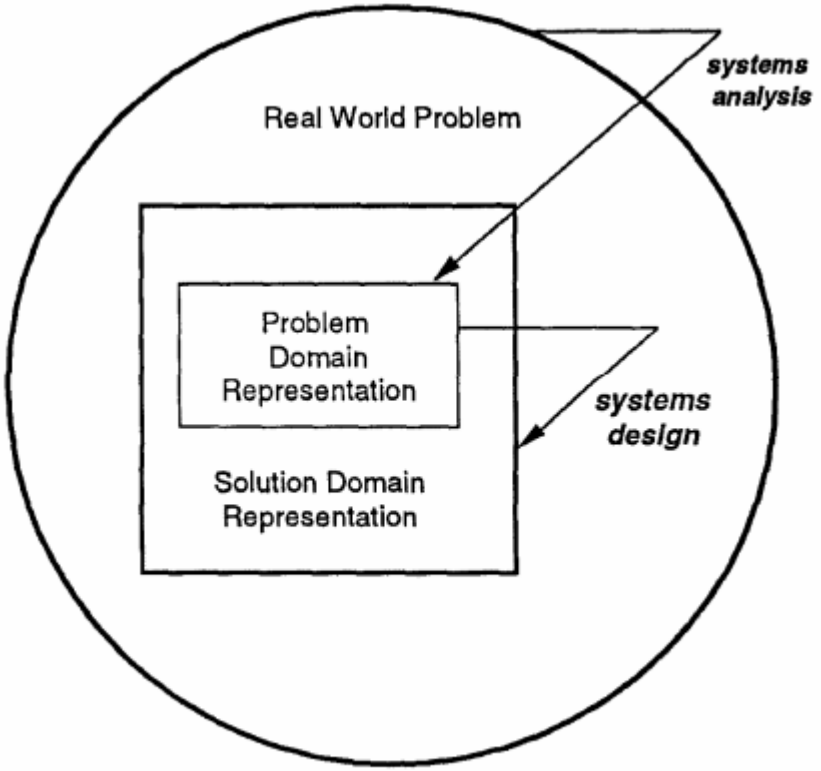
Stefan Henkler E-Mail: stefan.henkler@hshl.de

Faezeh Pasandideh E-Mail: faezeh.pasandideh@hshl.de


user requirements
textual requirements
analysis
descriptive model
Real world model
reusable component library
design + construction
computational model
constructive model
[Nerson1992]

3 (Requirement) Analysis



[Monarchi&Puhr1992]

- **Analysis** models the problem in the **problem domain** by identifying and specifying a relevant subset of the real world according to system requirements.
- **Design** models the solution in the **solution domain**, which often includes most elements of the problem plus additional solution specific elements.

Traffic Light example

- **User Requirements (WHAT is needed?)**
- Cars must stop when a pedestrian presses the button
- The pedestrian green light must stay ON for at least **15 seconds**
- **Descriptive Model (Analysis)**
- This is a **conceptual view of the system**, showing:
- Pedestrian button
- Car traffic lights (red/green)
- Pedestrian lights (red/green)
- Buzzer
- Connections to the Arduino

Traffic Light example

- **Constructive Model (Design / Implementation)**
- This is the **technical solution**, including:
  - Arduino Uno hardware
  - Wiring diagram and LED connections
  - Arduino program (code)
  - Timing logic using software
- **Component Library (Reusable Parts)**
  - Standard LED control circuits
  - Button debouncing techniques
  - Timer libraries and delay management methods

Traffic Light example

Analysis (WHAT?)	Design (HOW?)
Understand user needs and system behavior	Decide technical implementation
Focus on **requirements**	Focus on **solution building**
What the system must do	How the system will work

- **Design** [IEEE-Std-610.12-1990][Taylor1959]:
  - (1) The process of defining the **architecture**, **components**, **interfaces**, and other characteristics of a system or component.
  - (2) The result of the process in (1). The process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization.
- **Design description** [IEEE-Std-610.12-1990]:
  - A document that describes the design of a system or component. Typical contents include system or component **architecture**, **control logic**, **data structures**, **input/output-formats**, **interface descriptions**, and **algorithms.**

Traffic Light example: Design Description

Architecture

Arduino microcontroller acts as the **central controller**, connected to LEDs, a button, and a buzzer.

Components

- **Car traffic lights:**
  - Red LED → Pin 9
  - Yellow LED → Pin 10
  - Green LED → Pin 11
- **Pedestrian traffic lights:**
  - Red LED → Pin 6
  - Green LED → Pin 7
- **Pedestrian button:**
  - Digital input Pin 2 (with pull-up resistor)

Traffic Light example: Design Description

HOCHSCHULE HAMM-LIPPSTADT

Data Structures

- enum TrafficState { CAR_GREEN, CAR_YELLOW, CAR_RED, PED_GREEN, PED_FLASH }
- unsigned long phaseStartTime (tracks timing of each state)
- bool buttonPressed (stores pedestrian request)

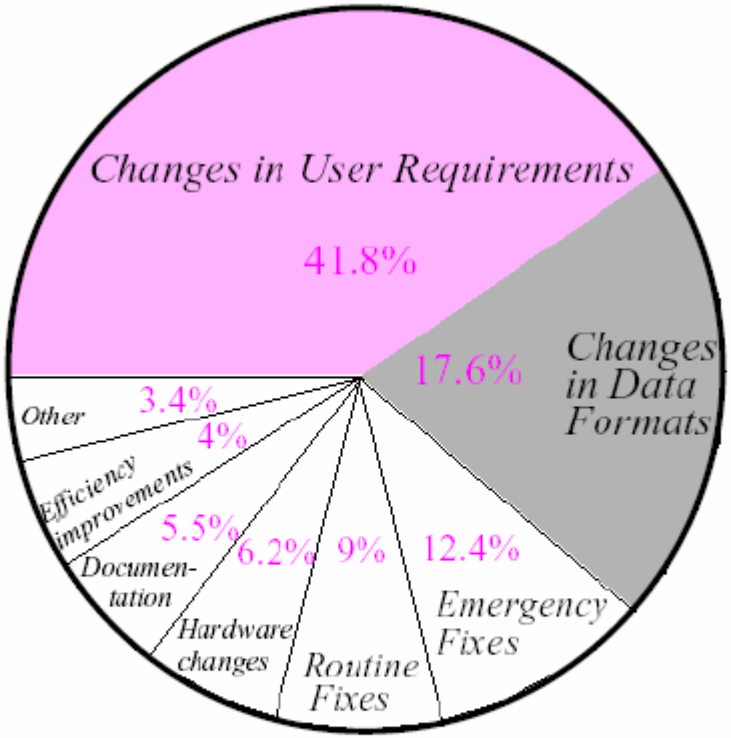
Control Logic

The system is implemented as a **finite state machine (FSM)** with 5 states:

- Each state represents a traffic phase
- Transitions are triggered by **time conditions and button input**
- Each phase has a defined duration (timing-controlled behavior)

- **First Law of Software Evolution**
  - A program that is used and that has an implementation of its specification reflects some reality, undergoes continual change or becomes progressively less useful.
- **First Law of System Engineering**
  - No matter where you are in the system life cycle, the system will change and the desire to change it will persist throughout the life cycle.

Software will be changed during maintenance



Breakdown of maintenance costs.
([Meyer1997] source [Lientz1980])

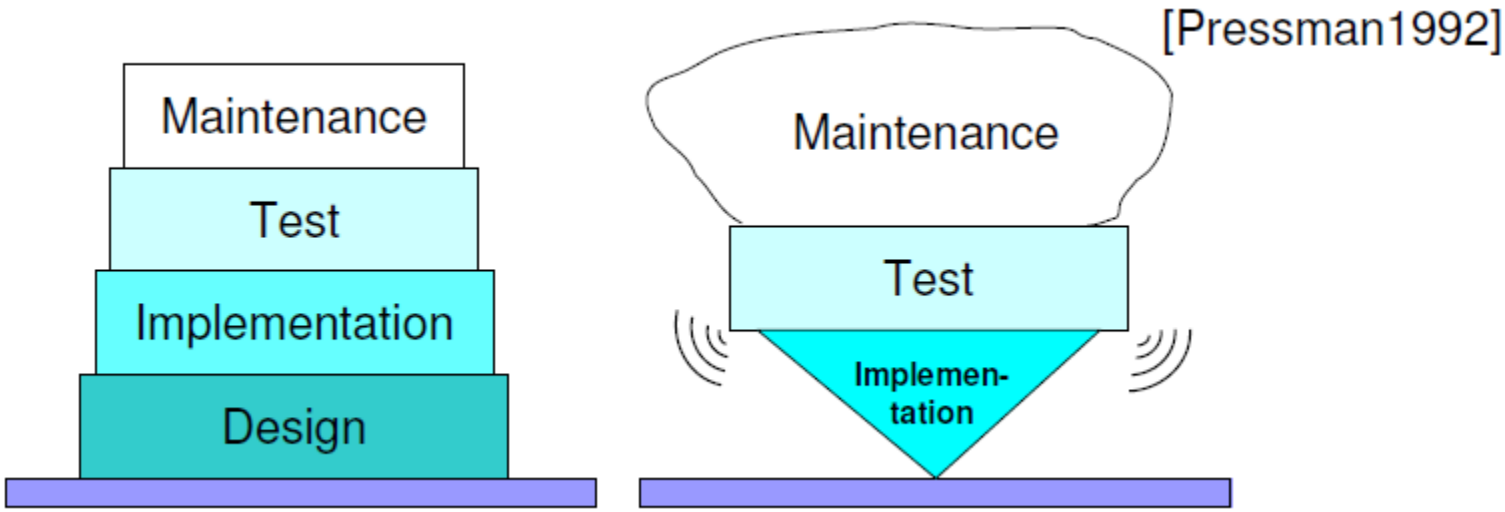
Traffic Light example: Design & Change

- **Version 1 — Original Requirement**
- Simple pedestrian traffic light
- Fixed **30-second cycle**
- **Change 1 — City Planner Update**
- *“We need a longer green phase during rush hours (7–9 AM and 5–7 PM).”*
- Requires adding **time-of-day logic**
- Needs a **real-time clock (RTC) module**
- **Change 2 — Accessibility Requirement**
- *“Add support for blind pedestrians.”*
- Add **audible beep (buzzer)**
- Add **vibration output (new hardware pin)**

Traffic Light example: Design & Change

- **Change 3 — Hardware Update**
- Switch from Arduino Uno to Arduino Mega.”
- More available pins
- Easy update **IF** design uses named constants like:
- #define PIN_RED 9
- **Problem:** If pin numbers are hardcoded everywhere → major rewrite needed
- A **good design** (with:
- named constants
- modular code
- clear state machine
) makes changes easy and safe.
- A **bad design** leads to **full system rewrites for small updates**.

Software Quality Requires Design



- Importance of software design: **quality**
  - Translate accurately requirements into product (accuracy)
  - Still stable systems for small changes (Stability)
  - Otherwise difficult to test (Testability)

Traffic Light example: Design Quality

Good Design (stable, testable):

- Each LED has a named constant: #define PIN_CAR_RED 9
- Each phase has a defined duration: const int CAR_YELLOW_MS = 3000
- State machine in one function: updateTrafficState()
- You can test each state independently

Bad Design (unstable, hard to test):

- Magic numbers everywhere: digitalWrite(9, HIGH); delay(3000);
- Logic mixed throughout the main loop
- Changing one timing value requires searching the whole file
- Almost impossible to add a new state without breaking others

HOCHSCHULE HAMM-LIPPSTADT

1. Introduction
2. Methods
3. Analysis
4. Design
5. Advanced Design Concepts
6. Discussion & Summary
7. Bibliography

HOCHSCHULE HAMM-LIPPSTADT

Historic Trends:

- Structured programming
  - Dijkstra 1968: Goto statement considered harmful
  - Keywords: top-down, functional decomposition, stepwise refinement, divide-and-conquer, …
  - SA/SD: late 1970s
- Object-oriented programming
  - Simular 67, smalltalk, C++, Object C, Eiffel, Ada, Lisp
  - Liskov, Guttag, Shaw 1970s: Abstract data type
  - Parnas 1972: Information hiding
  - Keywords: objects, classes, reusable components
  - OOA/OOD: late 1980s

Traffic Light example: Structured Vs. Object oriented

HOCHSCHULE HAMM-LIPPSTADT

▸ Structured approach (procedural code)

→ carGreenPhase()

→ carYellowPhase()

→ carRedPhase()

→ pedGreenPhase()

• Each function does one job, called in sequence

▸ Object-Oriented approach:

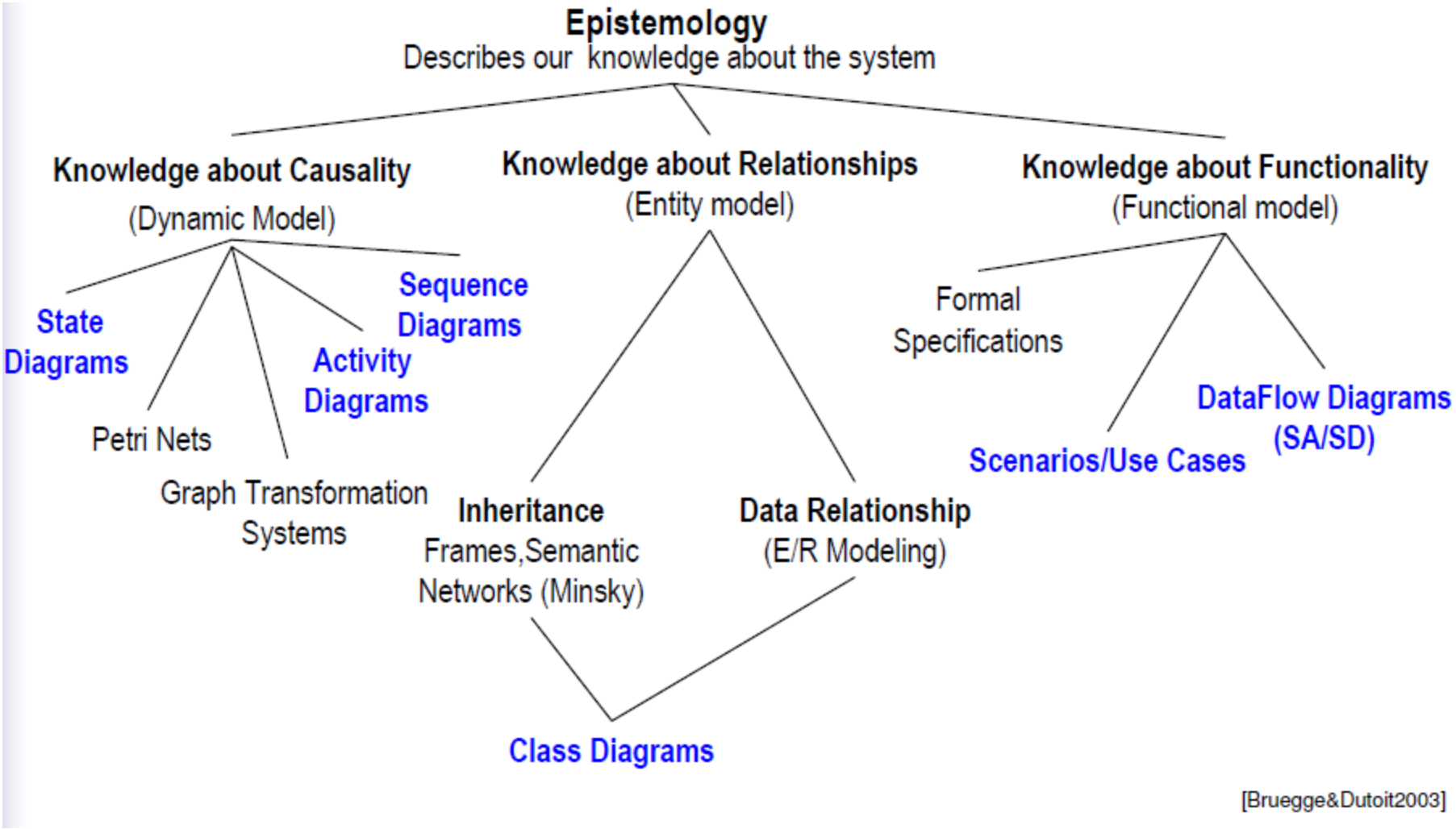
```
class TrafficLight {
private:
    int redPin;
    int yellowPin;
    int greenPin;

public:
    void setRed();
    void setYellow();
    void setGreen();
};
```

```
class PedestrianLight {
private:
    int redPin;
    int greenPin;
    Buzzer* buzzer;

public:
    void allowCross();
    void stopCross();
};
```

How to Describe Complex Systems?



[Bruegge&Dutoit2003]

HOCHSCHULE HAMM-LIPPSTADT

1. Introduction
2. Methods
3. Analysis
4. Design
5. Advanced Design Concepts
6. Discussion & Summary
7. Bibliography

HOCHSCHULE HAMM-LIPPSTADT

- Analysis should **model** aspects of the **real world** which are **relevant for the application** (objectives, requirements, application domain knowledge, requirements on the environment and requirements on the computer system).
- The model therefore should only describe the **required** or **existing structure** and **behavior of the application**.
- The **analysis model** is the **base for communication** between analysts, experts in the application domain and end users of the system.
- Main stake holders: end user, customer, analyst

Traffic Light example: Analysis Model

HOCHSCHULE HAMM-LIPPSTADT

- **Objectives:**

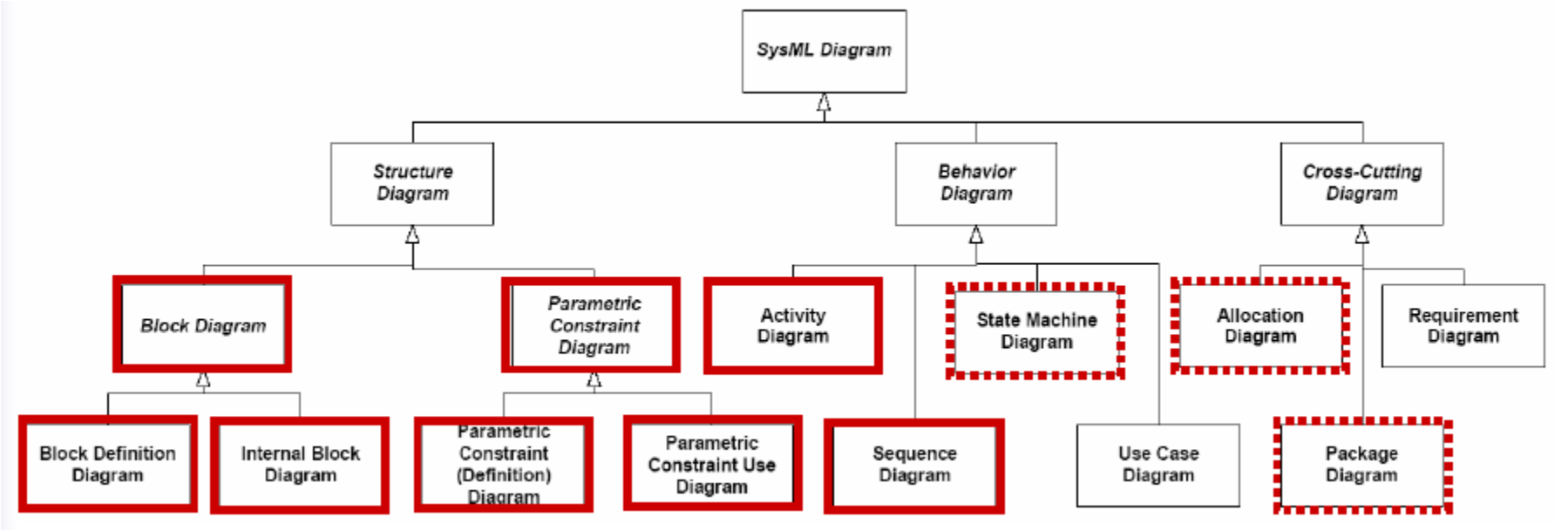
• Allow pedestrians to cross safely
• Minimize disruption to car traffic
• Comply with accessibility standards

- **Requirements:**

• The system SHALL stop car traffic within 5 seconds of button press
• The pedestrian green phase SHALL last at least 15 seconds
• The buzzer SHALL beep at 880Hz during the pedestrian green phase
• The system SHALL complete one full cycle in under 90 seconds

- **Environment** (external interactions):

• Pedestrian (presses button, crosses road)
• Driver (observes lights, stops/goes)
• Power supply (230V mains, regulated to 5V)
• Maintainer (adjusts timing, replaces LEDs)

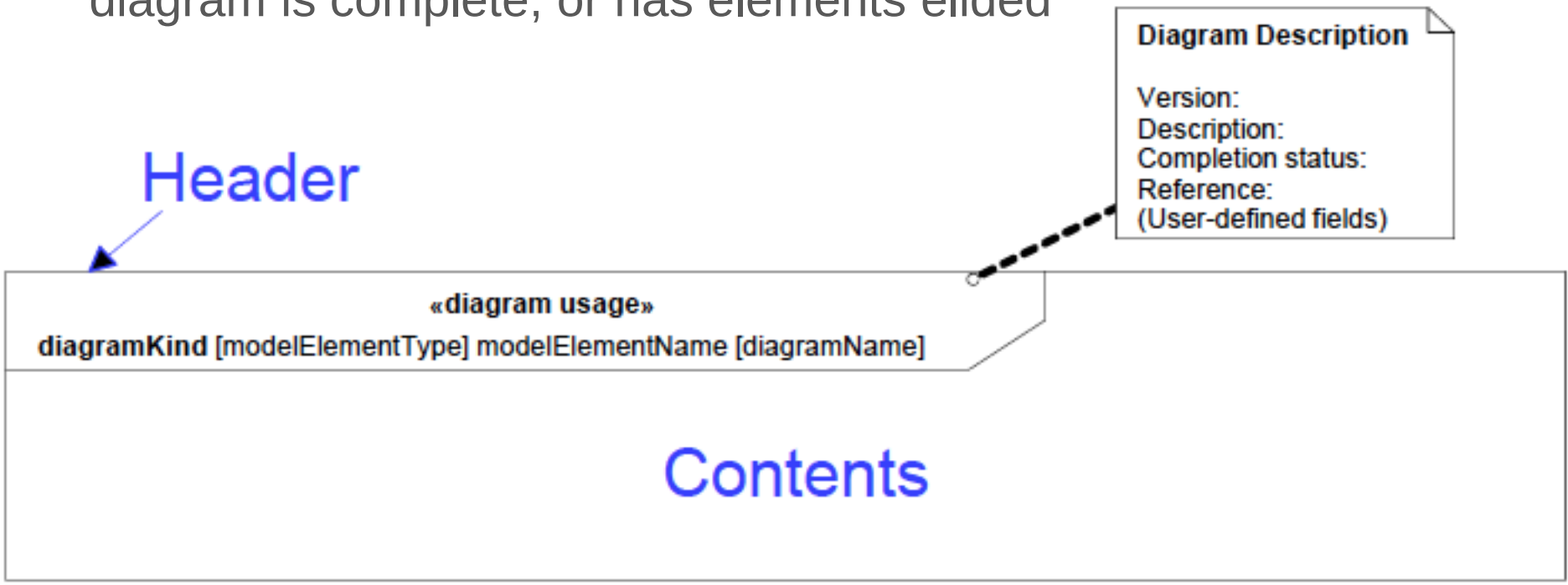


- (1) Block Diagrams
- (2) Parametric Constraint Diagram
- (3) Activity Diagrams
- (4) Sequence Diagrams

Before we start...

SysML Diagram Frames

- Each SysML diagram represents a model element [SysMLTutorial09]
- Each SysML Diagram must have a Diagram Frame
- Diagram context is indicated in the header:Diagram kind (act, bdd, ibd, sd, etc.)
  - Model element type (package, block, activity, etc.)
  - Model element name
  - User defined diagram name or view name
- A separate diagram description block is used to indicate if the diagram is complete, or has elements elided



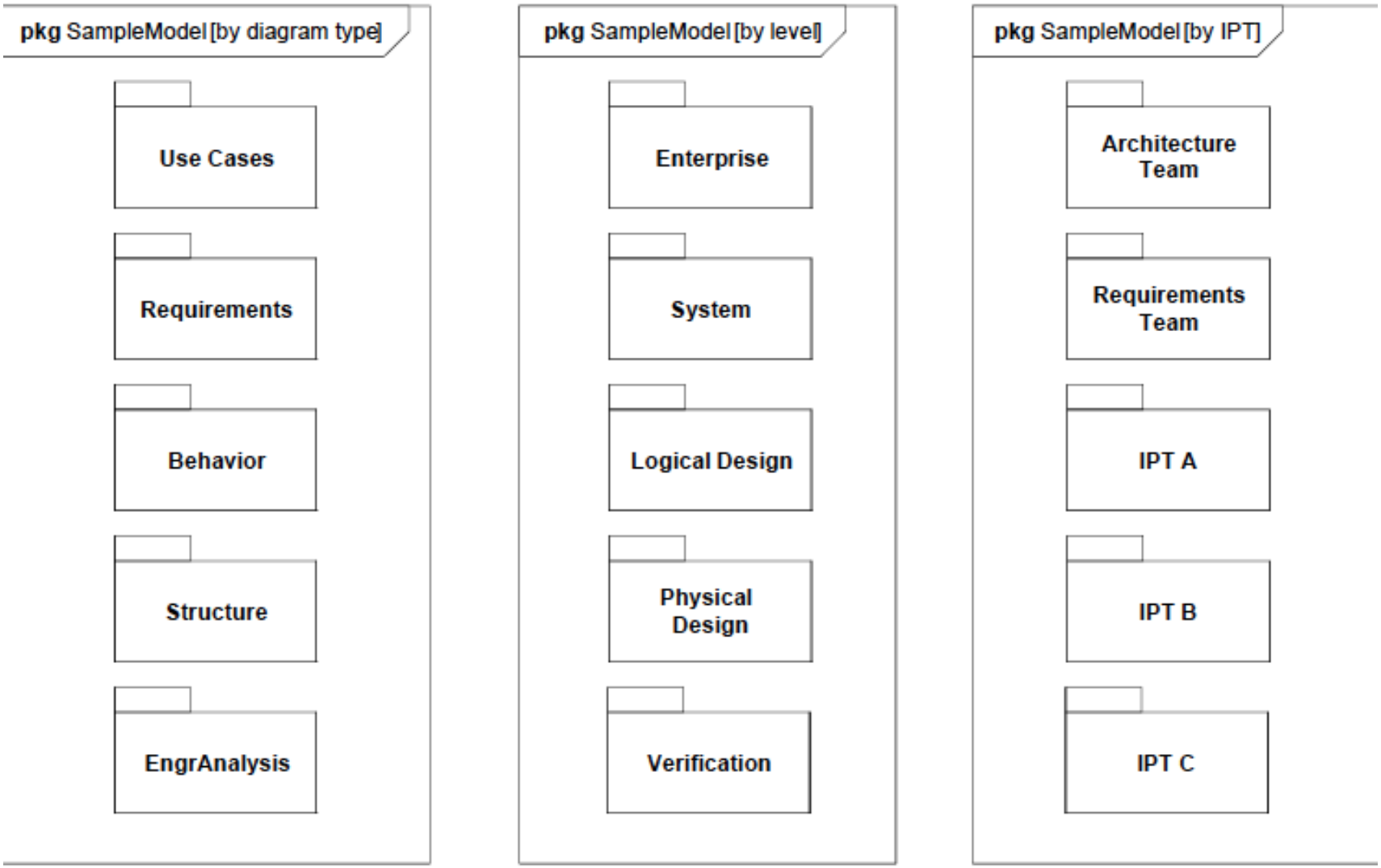
Before we start...

Package Diagram

- Package diagram is used to organize the model [SysMLTutorial09]
  - Groups model elements into a name space
  - Often represented in tool browser
  - Supports model configuration management (check-in/out)
- Model can be organized in multiple ways
  - By System hierarchy (e.g., enterprise, system, component)
  - By diagram kind (e.g., requirements, use cases, behavior)
  - Use viewpoints to augment model organization
- Import relationship reduces need for fully qualified name (package1::class1)

Before we start...

Package Diagram Example



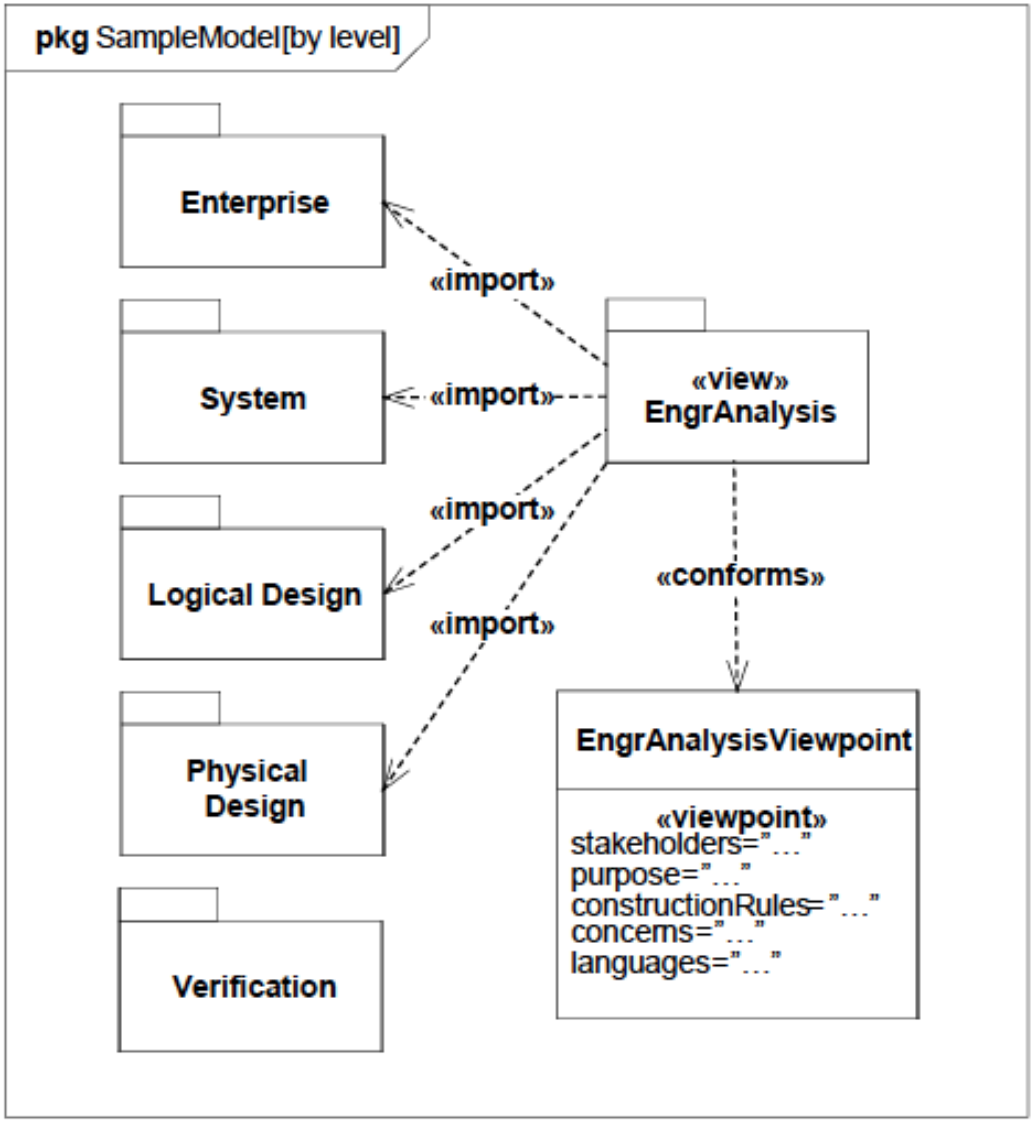
By Diagram Type

By Hierarchy

By IPT

Before we start

Package Diagram - Views



- Viewpoint represents the stakeholder perspective

- Engineer → cares about **performance**
- Manager → cares about **cost**

- View conforms to a particular viewpoint (actual diagram)
  - Imports model elements from multiple packages

The **View** (EngrAnalysis) collects information from different parts (Enterprise, System, etc.)

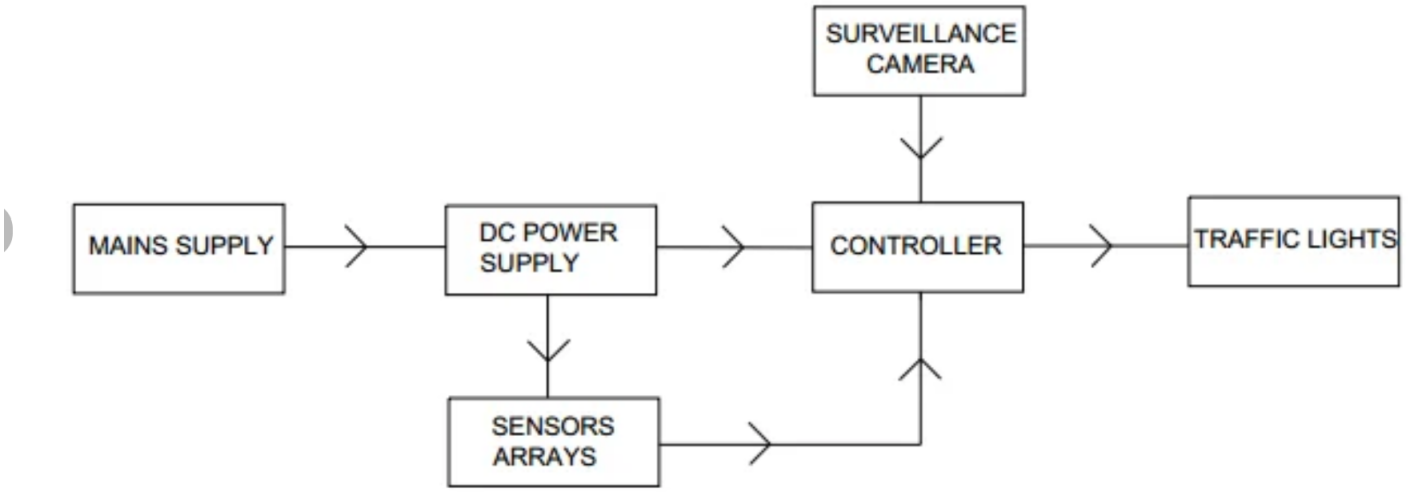
And it follows the rules of a Viewpoint (EngrAnalysisViewpoint)

What can/should be Analyzed?

- (1) Block Diagrams
  - Given structures and interfaces (Parts & how are they related)
- (2) Parametric Constraint Diagram
  - Dependencies/Constraints between given elements (math/logic constraints)
- (3) Activity Diagrams
  - Scenarios describing required activities (Steps & Workflows)
- (4) Sequence Diagrams
  - Required/Likely interaction scenarios (parts interactions)
- (5) State Machine Diagram
  - Complete state-dependent reactive behavior of given elements (events)

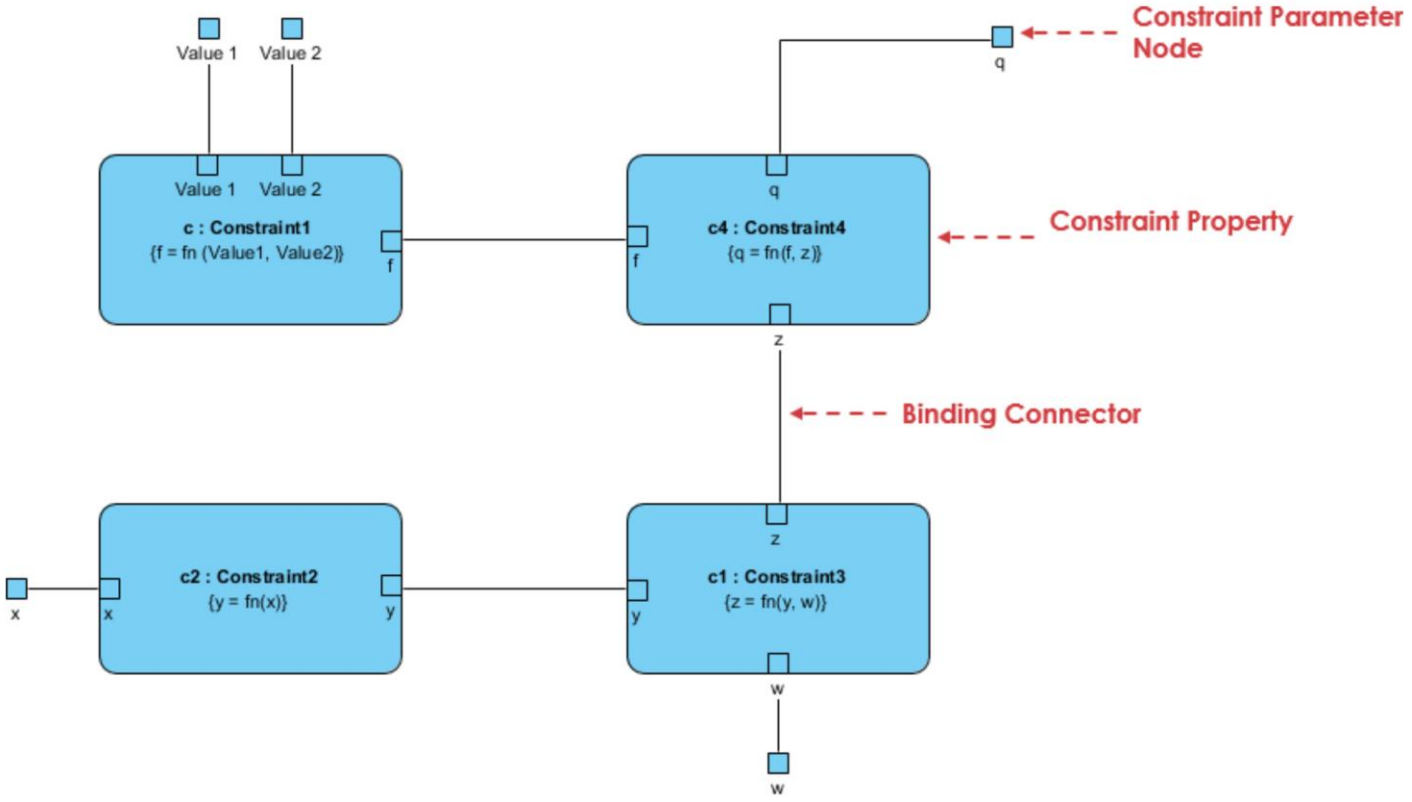
What can/should be Analyzed?

- (1) Block Diagrams
  - Given structures and interfaces (Parts & how are they related)



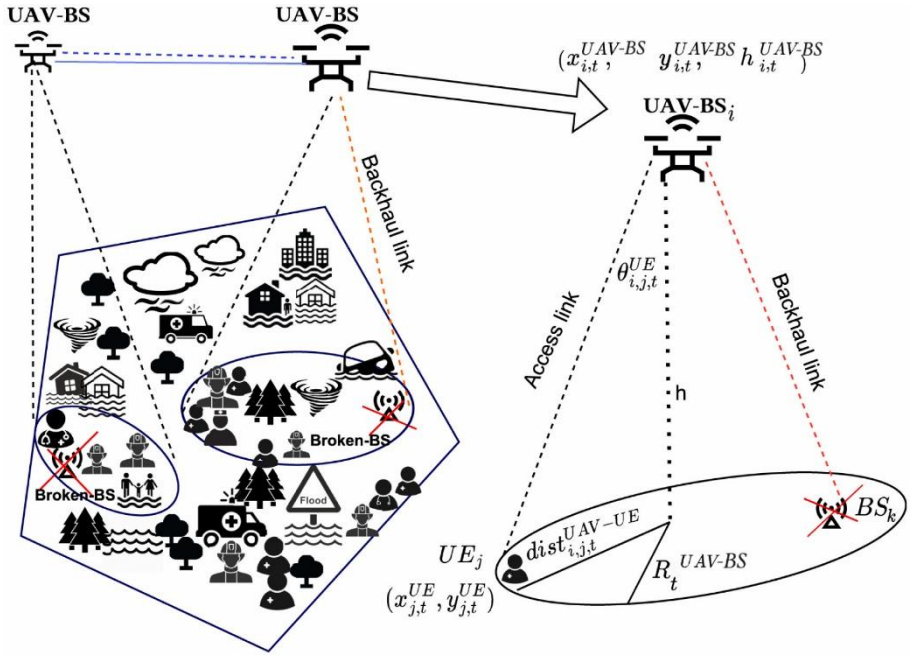
What can/should be Analyzed?

- (2) Parametric Constraint Diagram
  - Dependencies/Constraints between given elements (math/logic constraints)



What can/should be Analyzed?

(2) Parametric Constraint Diagram



Mixed-Integer Non-Linear Programming (MINLP)

$$f_1 = \sum_i^I \sum_t^T P_{i,t}^{\text{Total}}, \forall i,t \quad (1)$$

$$f_2 = -\sum_i^I \sum_j^J \sum_t^T I_{i,j,t}^{\text{UE}}, \forall i,j,t \quad (2)$$

$$\min f = w_1 \cdot f_1 + w_2 \cdot f_2 \quad (3)$$

$$w_1 + w_2 = 1 \quad (4)$$

$$V_{i,t}^{\text{Vertical}} = \frac{h_{i,t}^{\text{UAV-BS}} - h_{i,t-1}^{\text{UAV-BS}}}{dt}, \forall i,t \quad (5)$$

$$V_{i,t}^{\text{X}} = \frac{x_{i,t}^{\text{UAV-BS}} - x_{i,t-1}^{\text{UAV-BS}}}{dt} + V_{i,t}^{Wind} cos\left(\theta_t^{Wind}\right) I_{i,t}^{\text{UAV-BS}}, \forall i,t \quad (6)$$

$$V_{i,t}^{\text{Y}} = \frac{y_{i,t}^{\text{UAV-BS}} - y_{i,t-1}^{\text{UAV-BS}}}{dt} + V_{i,t}^{\text{Wind}} sin\left(\theta_t^{\text{Wind}}\right) I_{i,t}^{\text{UAV-BS}}, \forall i,t \quad (7)$$

$$V_{i,t}^{\text{Horizontal}} = \sqrt{\left(V_{i,t}^{\text{X}}\right)^2 + \left(V_{i,t}^{\text{Y}}\right)^2}, \forall i,t \quad (8)$$

$$V_{i,t} = \sqrt{\left(V_{i,t}^{\text{Vertical}}\right)^2 + \left(V_{i,t}^{\text{Horizontal}}\right)^2}, \forall i,t \quad (9)$$

$$x_{i,t}^{\text{UAV-BS}} = x_{i,t}^{\text{UAV,Initial}} \cdot I_{i,t}^{\text{UAV-BS}}, \forall t = 0, i \quad (10)$$

$$y_{i,t}^{\text{UAV-BS}} = y_{i,t}^{\text{UAV,Initial}} \cdot I_{i,t}^{\text{UAV-BS}}, \forall t = 0, i \quad (11)$$

$$h_{i,t}^{\text{UAV-BS}} = h_{i,t}^{\text{UAV,Initial}} \cdot I_{i,t}^{\text{UAV-BS}}, \forall t = 0, i \quad (12)$$

What can/should be Analyzed?

- (3) Activity Diagrams
  - Scenarios describing required activities (Steps & Workflows)

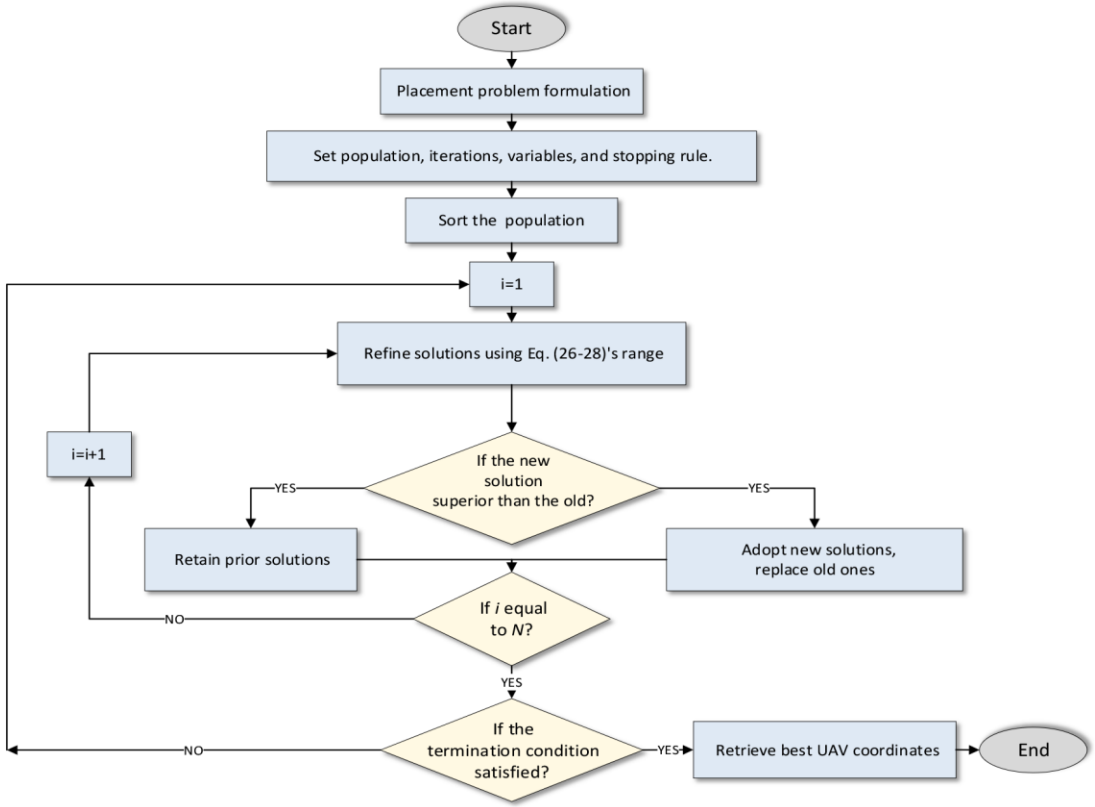
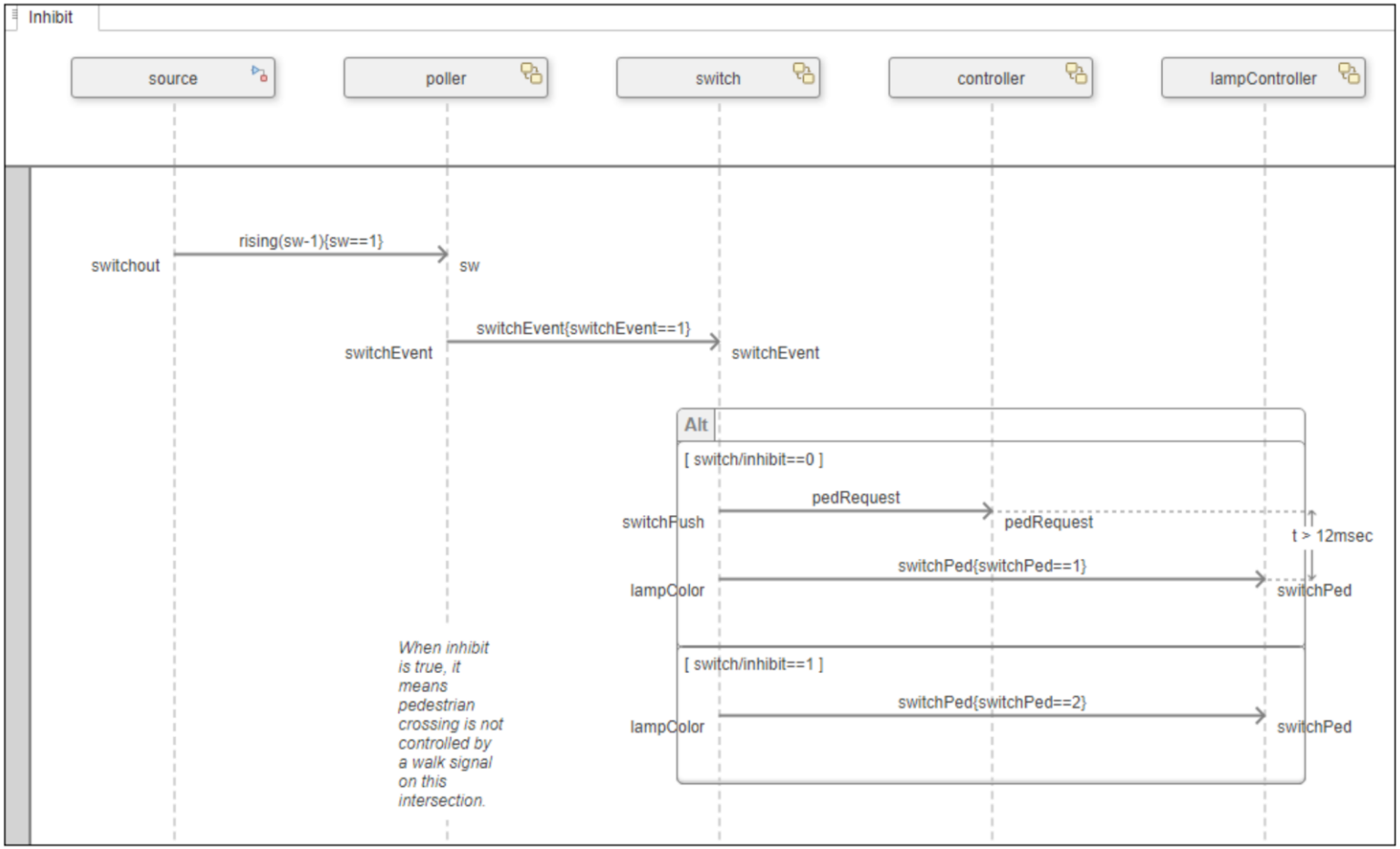


Figure 4.1 – JAYA algorithm for MINLP UAV-BS placement optimization problem.

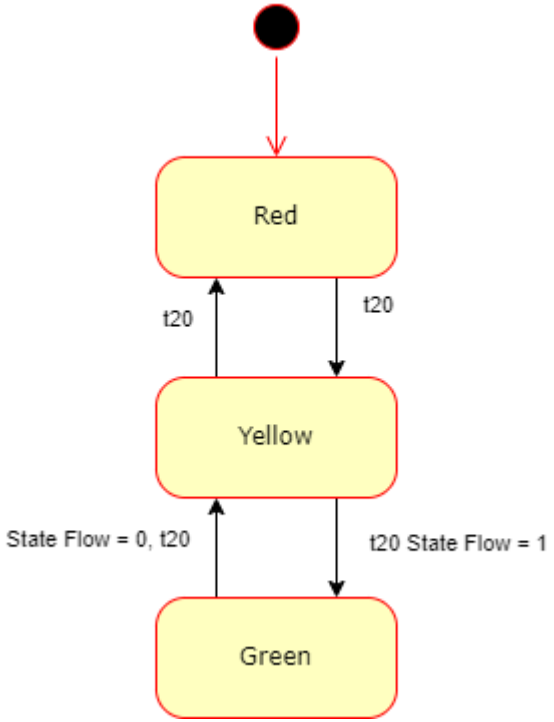
What can/should be Analyzed?

- (4) Sequence Diagrams
  - Required/Likely interaction scenarios (parts interactions)



What can/should be Analyzed?

- (5) State Machine Diagram
  - Complete state-dependent reactive behavior of given elements (events)



(1) Block Diagram

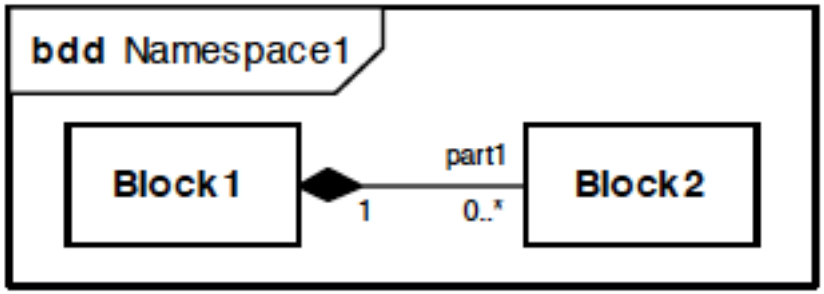
- A **Block** is a modular unit of system that **encapsulates** its contents, which include attributes, operations and constraints.
  - Blocks can be connected to other Blocks to form **composite structures**, and can be decomposed into **parts** to expose internal structures. (class in a code)
- Each **part** is specified by a block with its own properties, ports, and internal structure (an instance a bigger block)
  - a uniform set of elements is used to represent multiple levels of a system hierarchy. (**blocks are used for definitions, and parts are used for applications**)
- **Application:**
  - The SysML **block** model can be used throughout **all phases of system specification and design**, and can be applied to many different kinds of systems.
  - These **systems may be logical or physical, and may include software, hardware or human organizations**.
- A block is an unified concept for describing the structure of:
  - System, Hardware, Software, Data, Procedure, Facility, Person, …

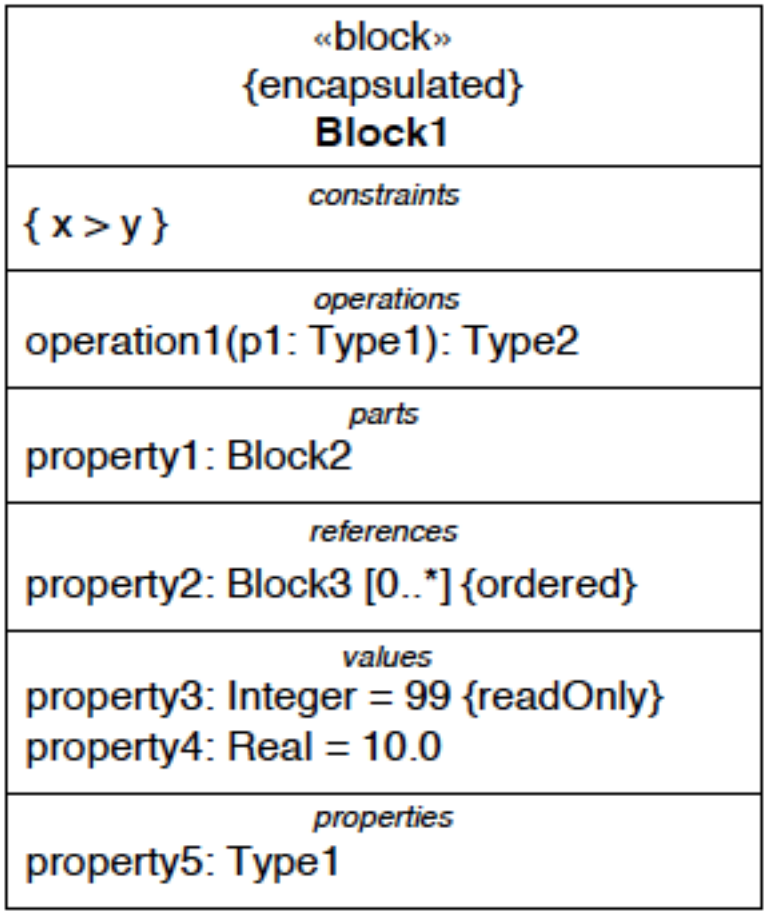
Block Definition Diagram

- Used to represent block definitions
- follows the graphical conventions of a **UML class diagram.**
- Showing blocks, their properties and their relationships.

Block

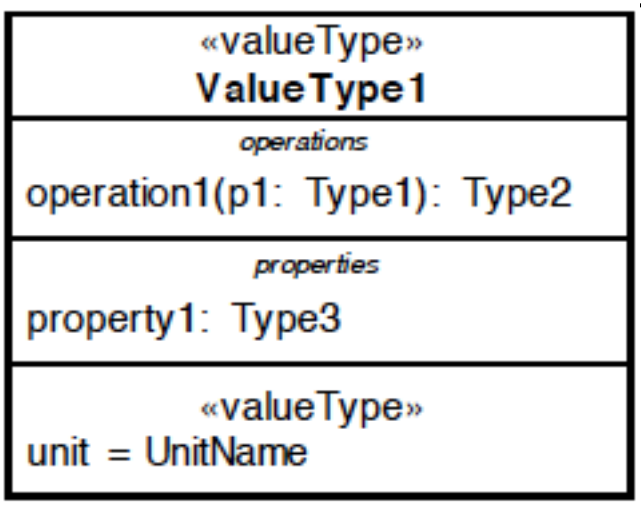
- A SysML Block defines a **collection** of **features** to describe a system or other element of interest.
- SysML blocks are **based** on UML **classes** - extended by UML composite structures
- **Properties** are the **basic structural characteristics** of blocks.
  - Properties may be of several types: Value properties, part properties, reference properties





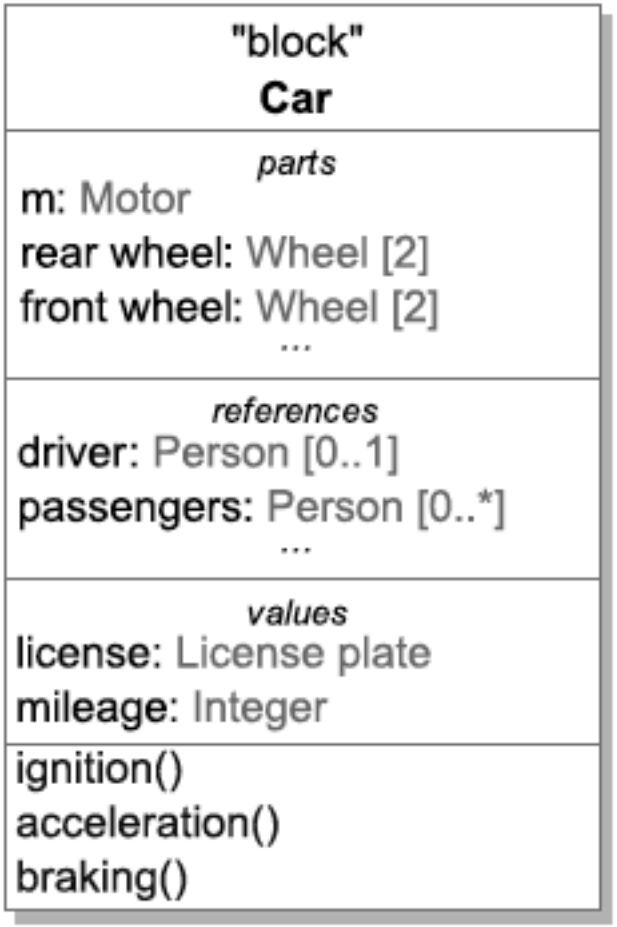
Block Property

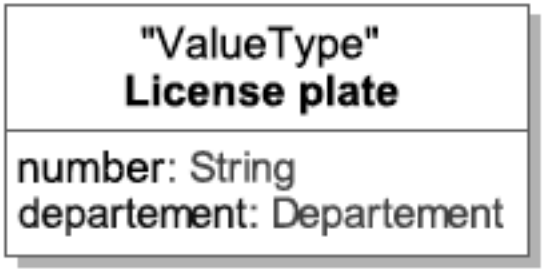
- **Value** (measurable charactresitcs) properties describe quantifiable characteristics in terms of value types (**range of values**, dimensions and optional units)
  - To define the types for value properties, SysML offers value types.
  - A SysML ValueType defines values that may be used within a model.
  - SysML value types are based on UML data types
- **Part** properties describe the decomposition hierarchy of the block in terms of other blocks.
  - A Block **is used** in the **context** of the enclosing block (composite) (e.g. left-front:wheel)
- **Reference** properties describe relations / association or simple aggregation with other blocks
  - Part **is not owned** by the enclosing block (no composition) (e.g. components are aggregated into logical subsystem)



Example

- the value type “License Plate” makes it possible to create a type that can be reused in the block “Car”



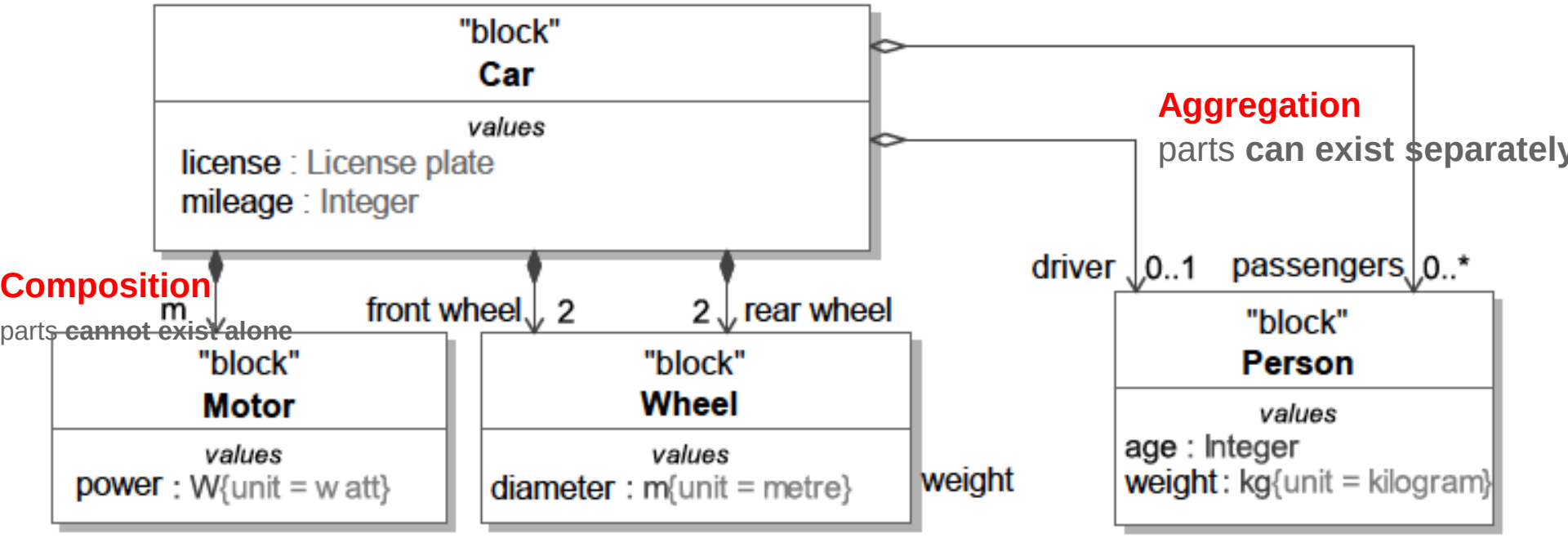


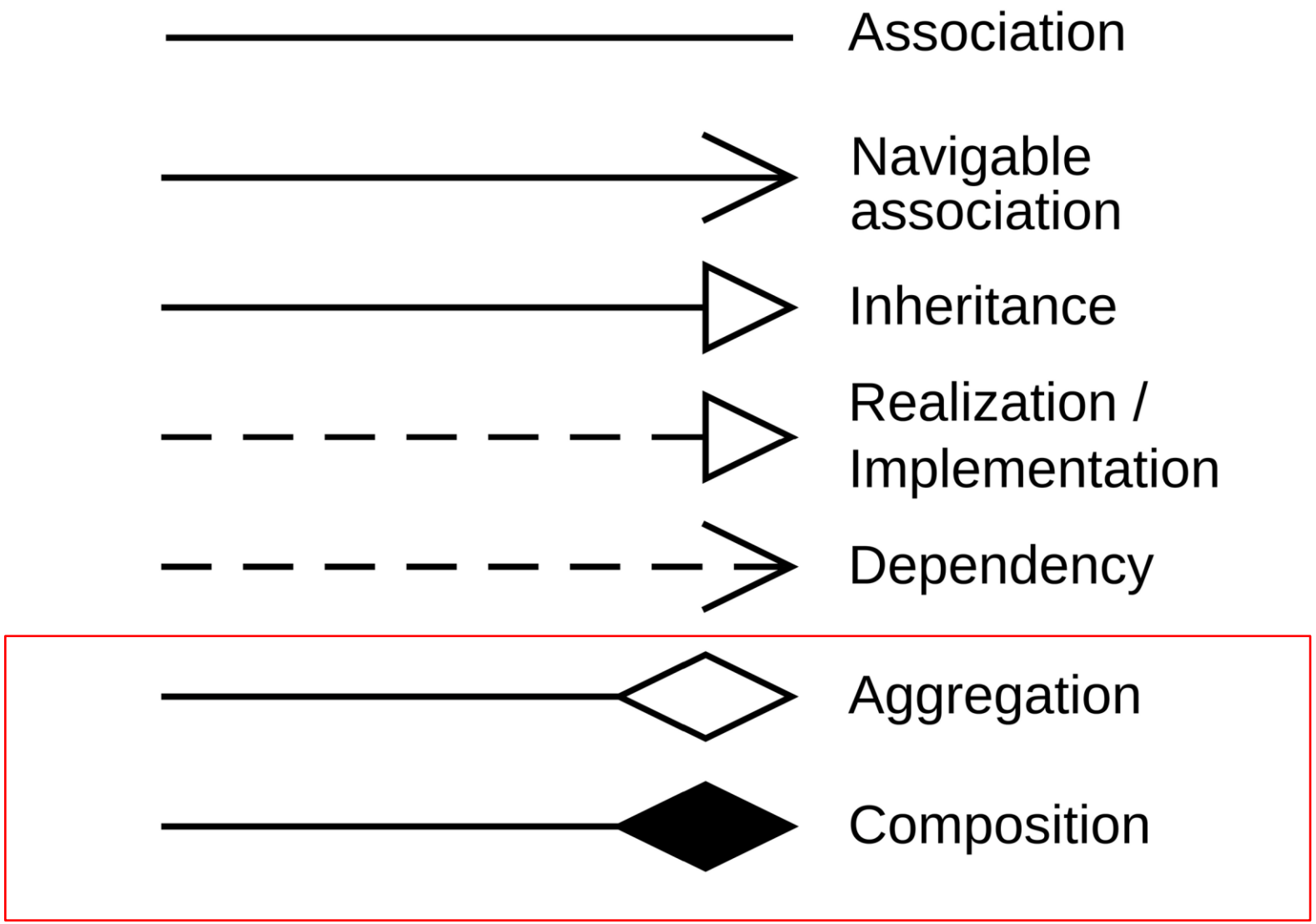
- number → String (e.g., "AB-1234")
- department → Department
- (e.g., region or city code)

Relations between Blocks

Association

- Two containment types: Aggregation, Composition (is made of…(as in UML)
- Static, enduring relation between two blocks
- Multiplicity should appear at each of its two ends
  - Specifies, as an interval, the number of instances that can participate in a relation with an instance of the other block in the context of this association
- An unidirectional association has an arrow pointing



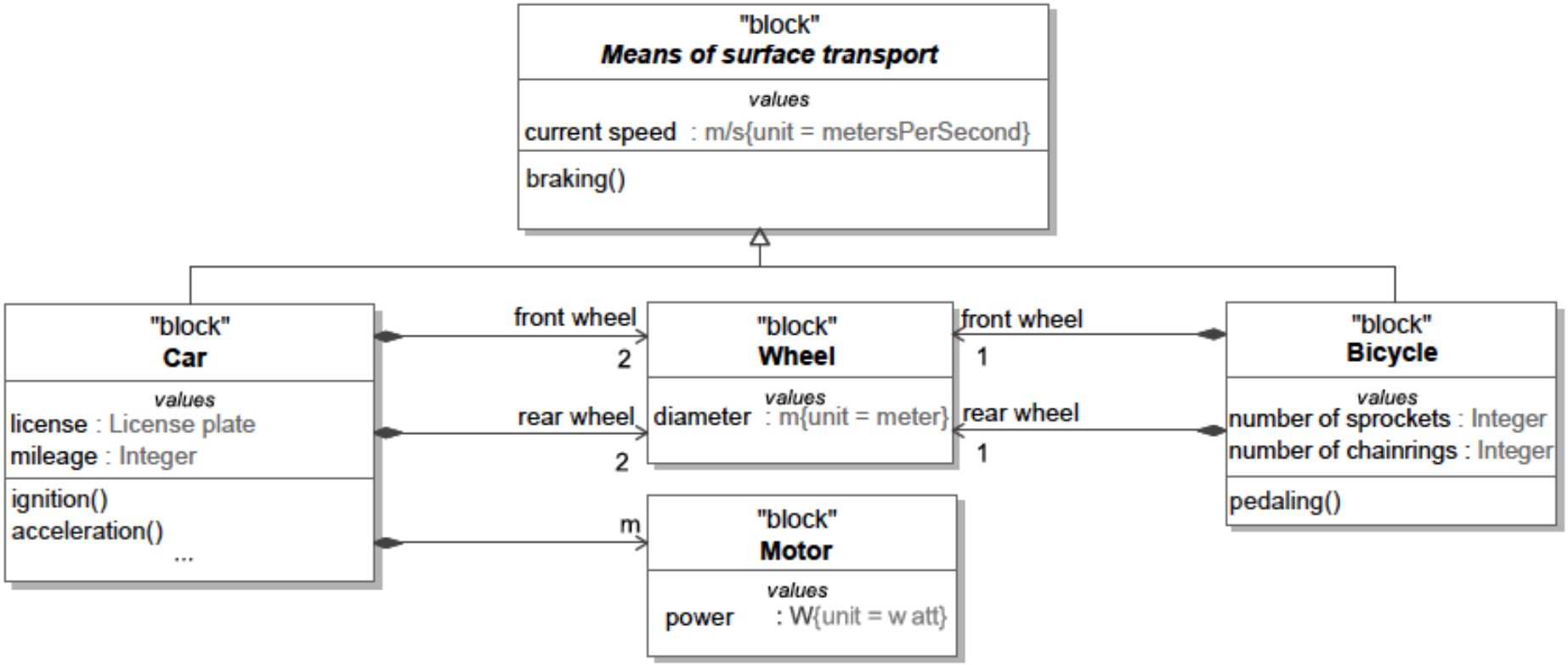

Association
Navigable association
Inheritance
Realization / Implementation
Dependency
Aggregation
Composition

Relations between Blocks

Generalization

- Blocks can be organized in a classification hierarchy
- Intended to factorize properties that are common to several blocks (values, parts, etc) in a generalized block

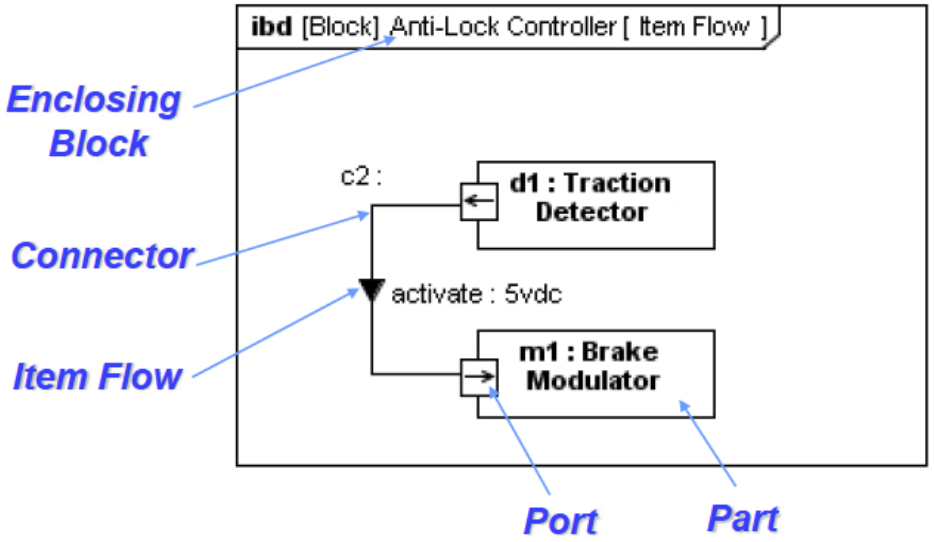
- Generalization (is a Type of ..). Specialized blocks “inherit” the properties of the generalized block and may have specific additional properties



Internal Block Diagram

Specify Interconnection of Parts

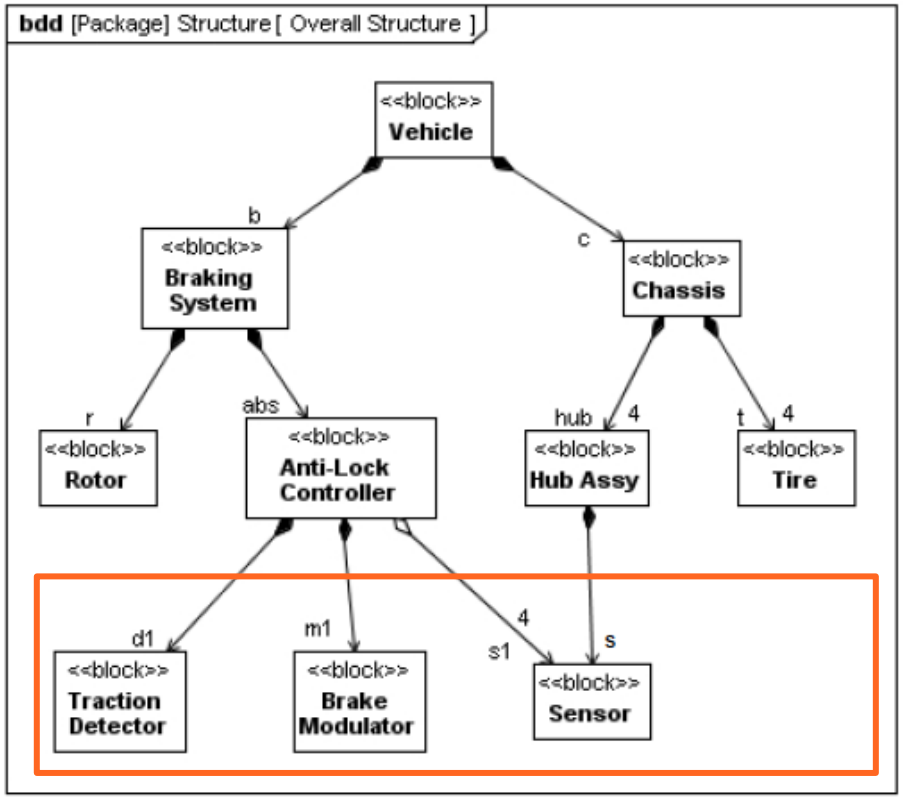
- The internal block diagram (ibd) describes the internal structure of a block in terms of parts, ports and connectors.
- Used to show the internal structure of a block and follows the graphical conventions of a **UML composite structure diagram** showing internal structure (**parts**, ports and connectors) of the subject block.
- note that we can represent several levels of decomposition in a single ibd.

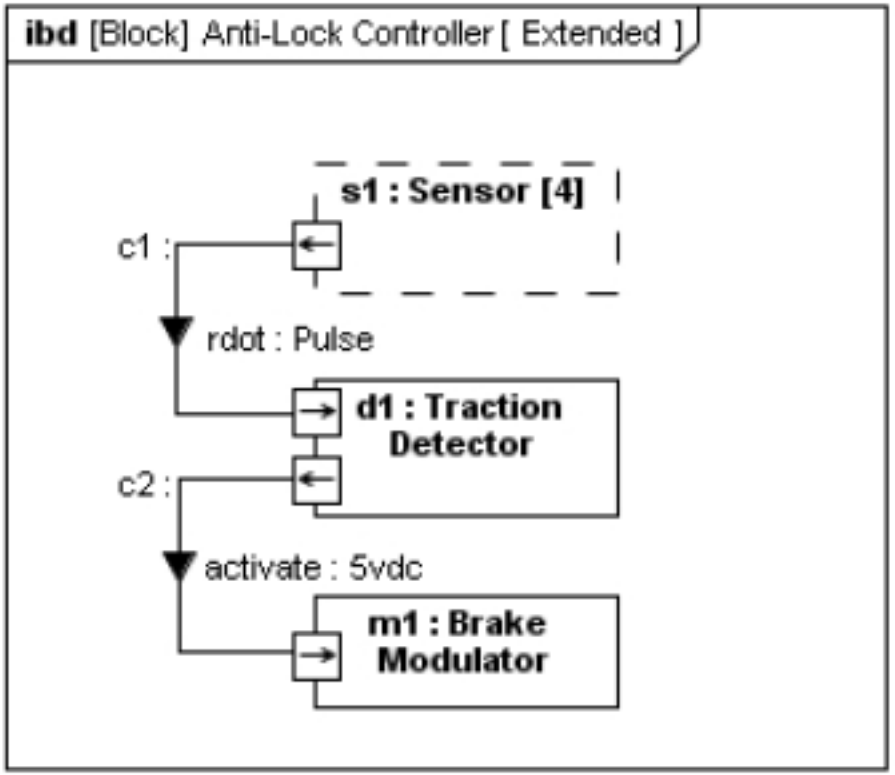


[SysMLTutorial09]

Internal Block Diagram

- A **composition** relation in a bdd can be represented with an ibd
- Each end of the composition relationship that exists in the bdd is **presented** as a **block** (known as a part) in the framework of the ibd
  - Part name is of the form: part_name: block_name [multiplicity]
- The multiplicity (1, by default) can also be represented in the upper right corner of the rectangle
- **Associations** and **aggregations** that are „outside“ the encompassing block are represented in a similar way to compositions, except that the line surrounding the block is **dashed**





Using Blocks – all together

Specify Hierachies and Interconnections

- Based on UML Class from UML Composite Structure
  - Supports unique features (e.g., flow ports, value properties)

- **Block definition diagram** describes the relationship among blocks (e.g., composition, association, specialization)
- **Internal block diagram** describes the internal structure of a block in terms of its properties and connectors
- **Behavior** can be allocated to blocks

[SysMLTutorial09]

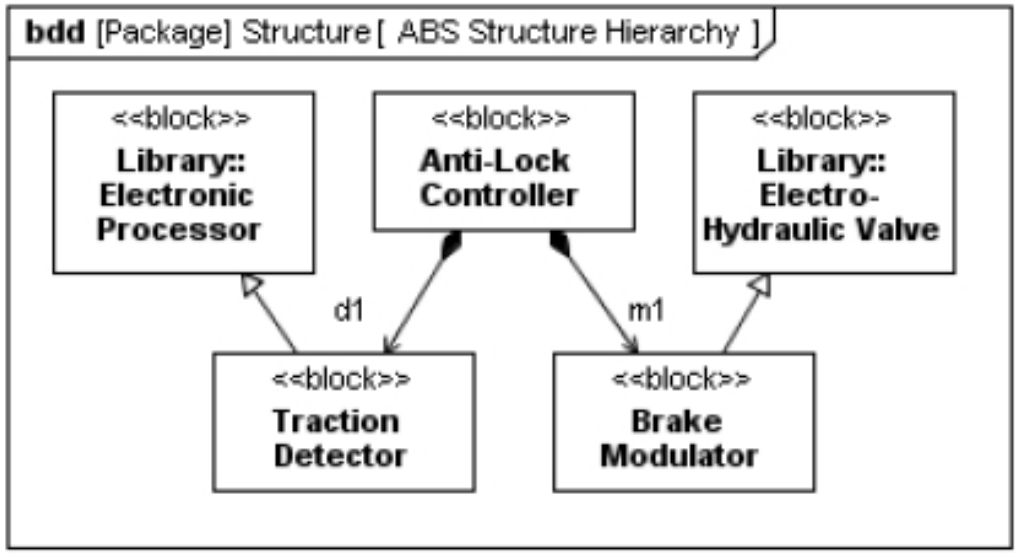
HOCHSCHULE HAMM-LIPPSTADT

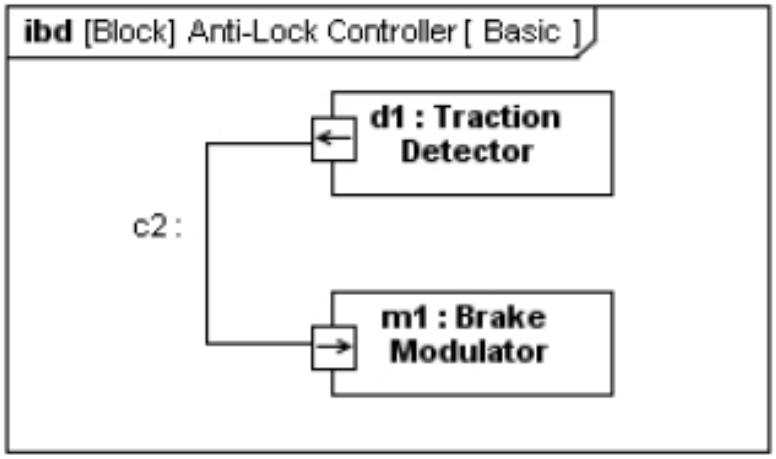
Definition

- Block is a definition/type
- Captures properties, etc.
- Reused in multiple contexts

Usage

- Part is the usage of a block in the context of a composing block
- Also known as a role



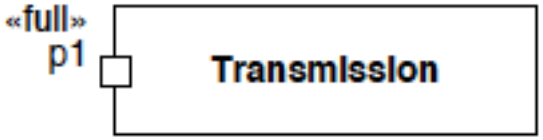


[SysMLTutorial09]

Ports

- The main motivation for specifying ports and flows is to enable design of modular, **reusable blocks** with **clearly defined** ways of **connecting** and **interacting** with their context of use.
- extends UML ports to support
  - nested ports
  - extends blocks to support flow properties
- Ports can be typed by blocks that support operations, receptions, and properties as in UML
- Two kinds of ports with respect of owning
  - One which exposes features of the owning block or its internal parts (**proxy ports**)
  - another that supports its own features (**full ports**)





Ports

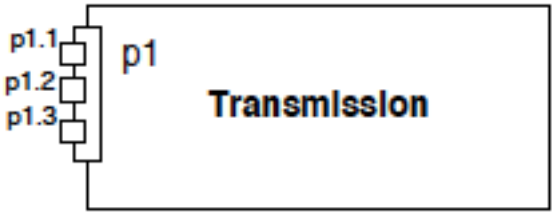
- Ports are points at which external entities can **connect to** and **interact** with a block in different or more limited ways than connecting directly to the block itself
- They are properties with a type that specifies features available to the external entities via connectors to the ports
  - including flow properties and association ends, as well as operations and receptions
- Port

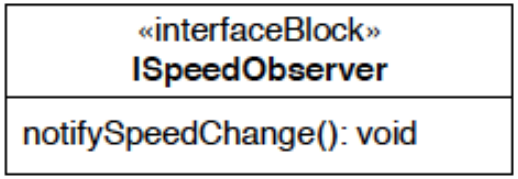


Ports with Flow Properties



- Nested Port
  - ports can type other ports
    - Via nested ports
    - Defined by interface blocks





Traffic Light example: Block Definition

- **Block**: TrafficController

- **constraints**: { cycleTime <= 90 }

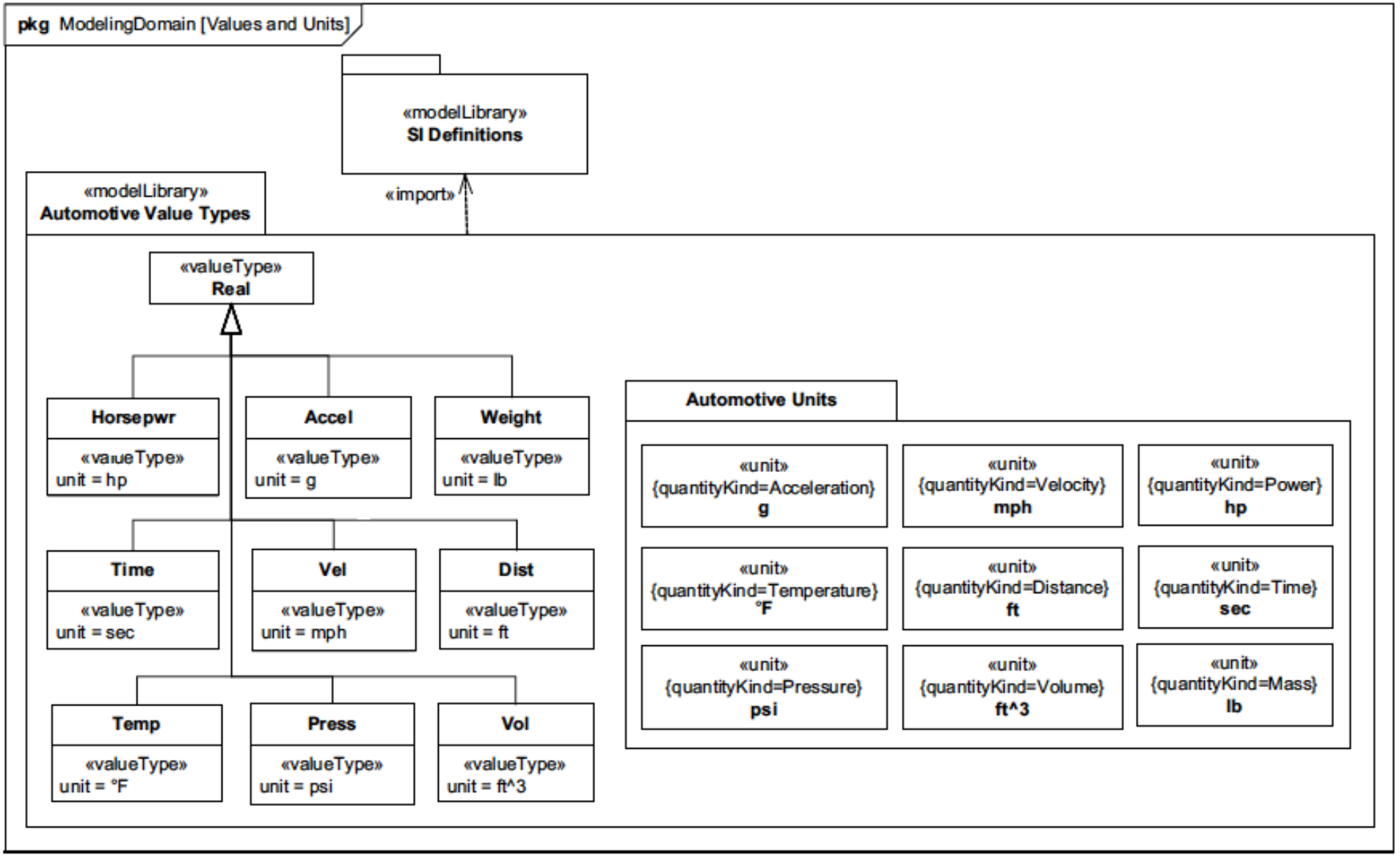
- **operations**: initialize(), updateState(), handleButtonPress()

- **parts**:
  - carLight : CarLight [1]
  - pedLight : PedestrianLight [1]
  - button : PushButton [1]

- **values**:
  - currentState : TrafficState
  - phaseStartTime : Integer
  - buttonPressed : Boolean

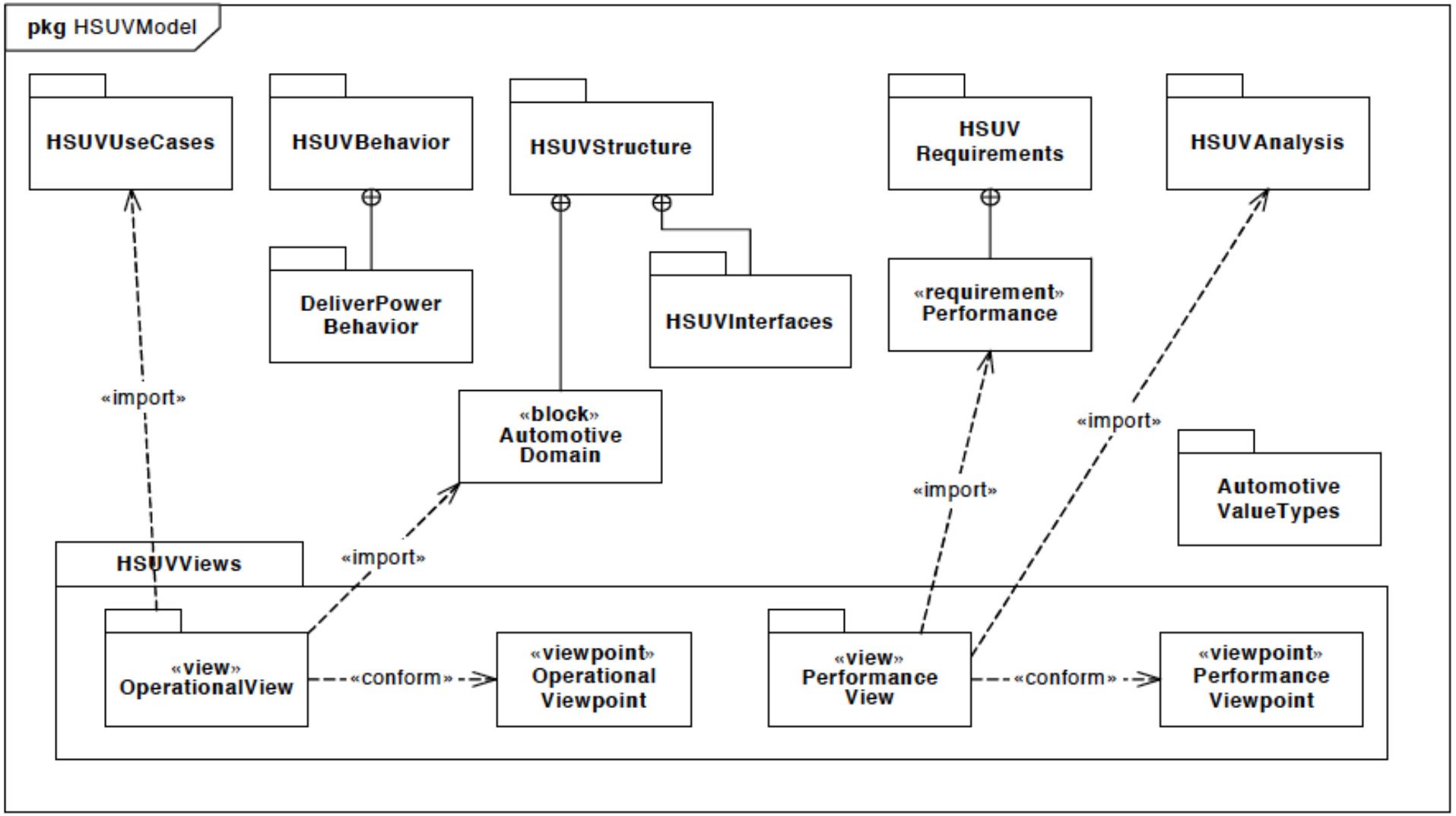
Automotive use case “Sample Problem” common “measurement language”

specification of units and valueTypes employed in the sample problem



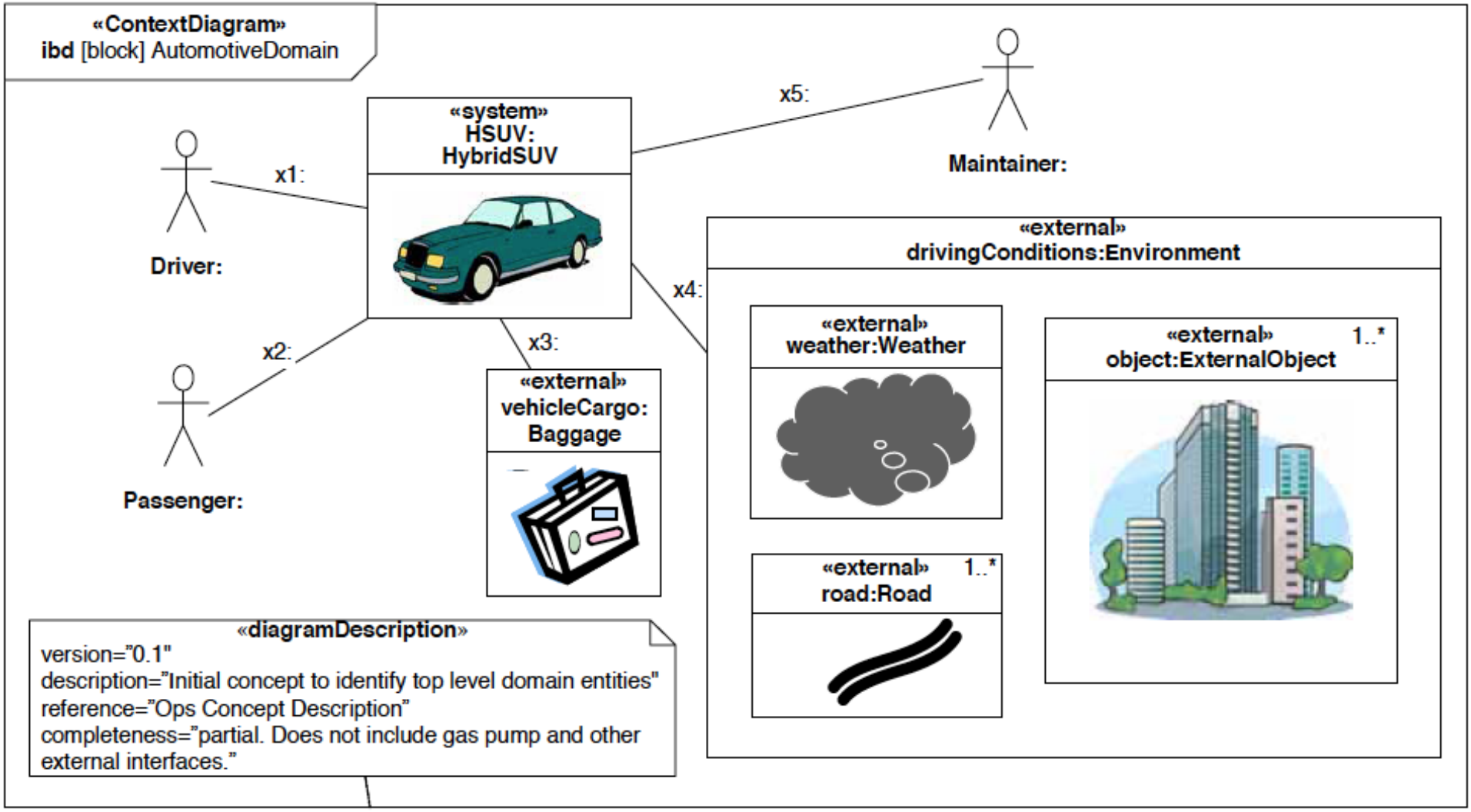
Sample Problem

Package Diagram



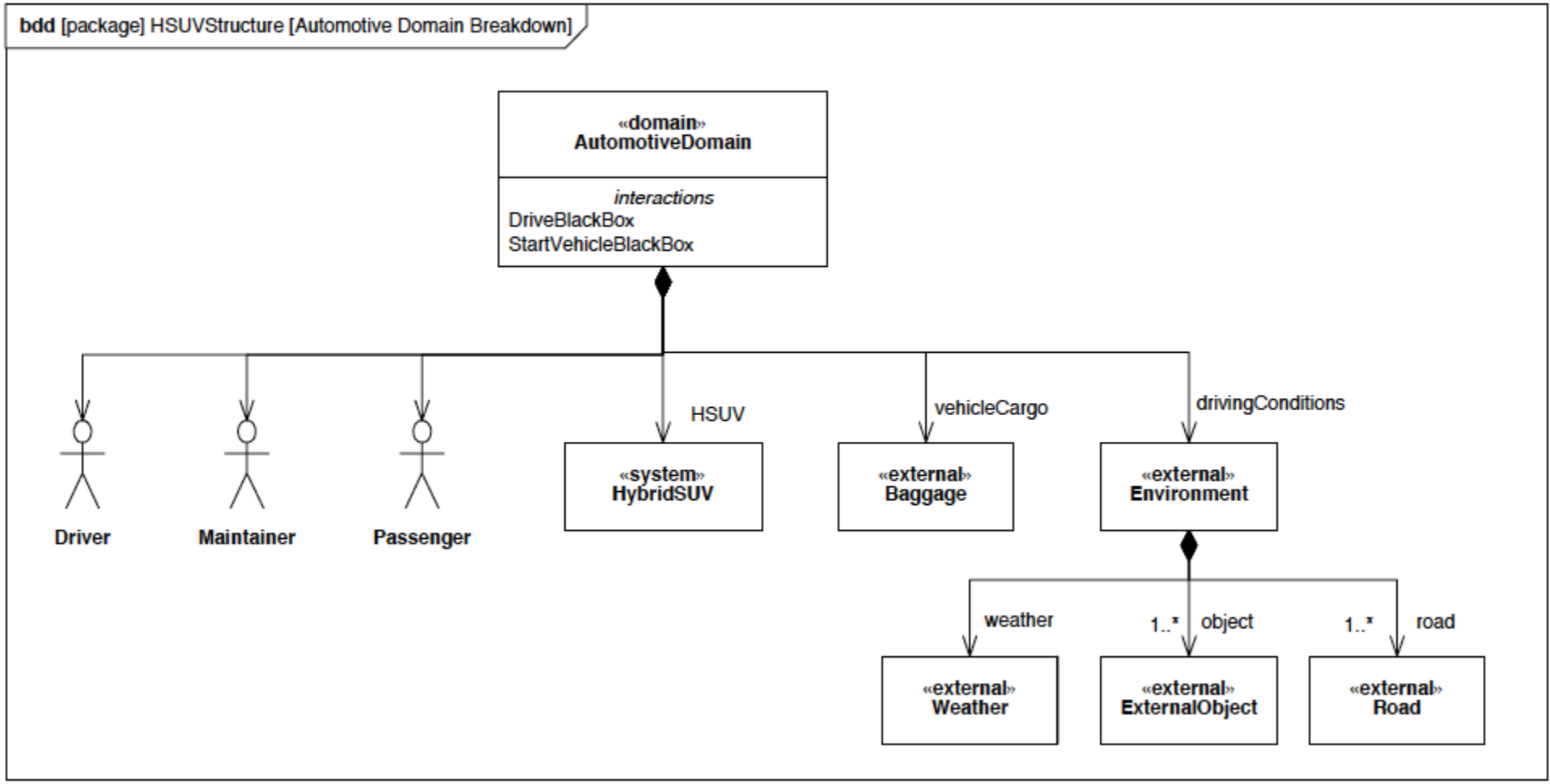
Sample Problem

Setting the context



Sample Problem

Block Definition Diagram – Automotive Domain



Homework

- Consider your Microcontroller project from last semester
  - Specify the blocks (bdd and ibd) on analysis level
    - Use paper and pen
  - In addition, if you have the possiblity, use a tool (e.g. draw.io)
- Readings
  - Tim Weilkiens, “Systems Engineering with SysML/UML” (see: https://learning.oreilly.com/library/view/systems-engineering-with/9780123742742/)
    - 4.5. Block diagrams (recap)
    - 4.6. Parametric block diagrams